

Serverless Web Application with AWS Lambda and API Gateway

Services Used

Amazon S3, CloudFront, API Gateway, AWS Lambda, Amazon DynamoDB, IAM

Description

This project demonstrates the deployment of a serverless web application on AWS. The application allows users to store and retrieve student data through a simple web interface. The frontend is hosted using Amazon S3 and delivered securely via CloudFront using HTTPS. Backend functionality is implemented using AWS Lambda, which is triggered through API Gateway. DynamoDB is used as a NoSQL database to store student records.

Definition

A serverless web application is an architecture where backend services are managed by the cloud provider, eliminating the need to manage servers. The application automatically scales and handles requests using services like Lambda and API Gateway.

Scope of the Project

- Create a frontend web application using HTML, CSS, and JavaScript
- Host static files on Amazon S3
- Use CloudFront to enable HTTPS and faster content delivery
- Create REST API using API Gateway
- Use AWS Lambda for backend logic
- Store and retrieve data using DynamoDB
- Configure CORS for frontend-backend communication
- Implement secure access using IAM roles

This Project Covers

- Serverless architecture design
- S3 static website hosting
- CloudFront distribution setup
- API Gateway REST API creation
- Lambda function development
- DynamoDB integration
- CORS configuration
- Debugging real-world issues (CORS, proxy integration, CloudFront AccessDenied)

Methodology

1. Serverless Architecture (Implemented Method)

- Uses S3, CloudFront, API Gateway, Lambda, DynamoDB
- No servers (EC2) are used
- Backend runs only when requests are made
- Highly scalable and cost-efficient

2. Traditional Server-Based Deployment (Alternative Method)

- Application could be deployed on EC2
- Requires server management and scaling
- Less efficient compared to serverless
- Not used in this project

AWS Services Used in the Project

1. Amazon S3

Definition: Amazon S3 is an object storage service used to store files.

Purpose: To host frontend static files.

Role: Stores index.html and script.js and serves them as a website.

2. Amazon CloudFront

Definition: CloudFront is a content delivery network (CDN).

Purpose: To deliver content securely using HTTPS.

Role: Distributes S3 content globally and improves performance.

3. API Gateway

Definition: API Gateway is a service to create REST APIs.

Purpose: To expose backend endpoints.

Role: Handles GET and POST requests and triggers Lambda functions.

4. AWS Lambda

Definition: Lambda is a serverless compute service.

Purpose: To run backend logic.

Role: Processes GET (fetch data) and POST (insert data) requests.

5. Amazon DynamoDB

Definition: DynamoDB is a NoSQL database service.

Purpose: To store application data.

Role: Stores student records with partition key.

6. IAM Role

Definition: IAM is used for access control.

Purpose: To give permissions to Lambda.

Role: Allows Lambda to access DynamoDB and API Gateway.

Project Application Details

Application Name

Student Data Application

Frontend Technology

- HTML
- CSS
- JavaScript (jQuery)

Backend Technology

- Python (Lambda)
- Boto3 (AWS SDK)

Main Features

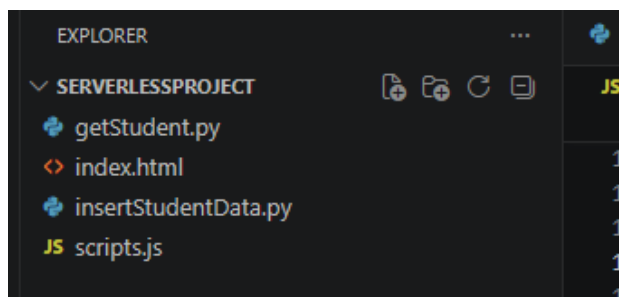
- Add student data
- View all student data
- Serverless backend processing
- Secure HTTPS access
- Real-time data storage

Implementation Steps:

Step 1: Create Frontend Application

Created index.html and script.js with form and table.

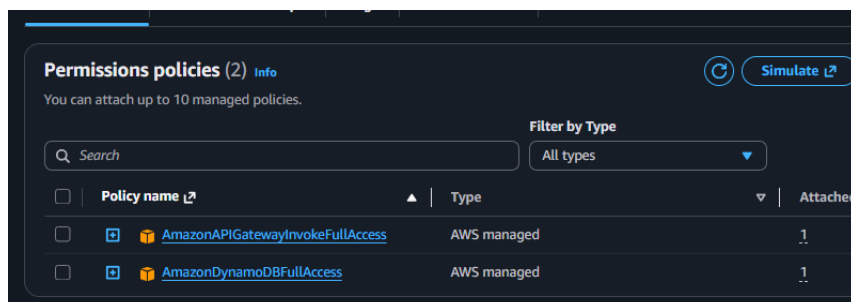
Purpose: To provide user interface for data input and display.



Step 2: Create IAM Role

- Created IAM role for **Lambda**
- Attached policies:
 - DynamoDB full access
 - API Gateway access

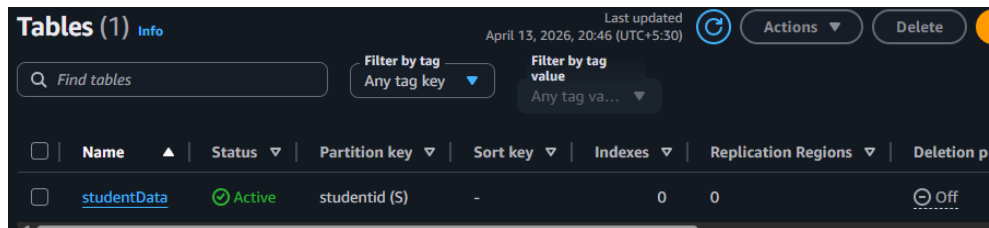
Purpose: To allow Lambda to interact with AWS services.



Step 3: Create DynamoDB Table

- Table name: **studentData**
- Partition key: studentid

Purpose: To store student records.



The screenshot shows the AWS IAM console interface for a table named 'studentData'. The table is listed in a table with columns: Name, Status, Partition key, Sort key, Indexes, Replication Regions, and Deletion protection. The 'studentData' table is shown with a status of 'Active', a partition key of 'studentid (S)', and a sort key of '-'. The table has 0 indexes and 0 replication regions. The deletion protection is set to 'Off'.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection
studentData	Active	studentid (S)	-	0	0	Off

Step 4: Create S3 Bucket

Bucket Purpose

Amazon S3 was used to host the frontend static website files.

Uploaded Files

- index.html
- scripts.js

Static Website Hosting

Enabled static website hosting on the S3 bucket.

Bucket Access

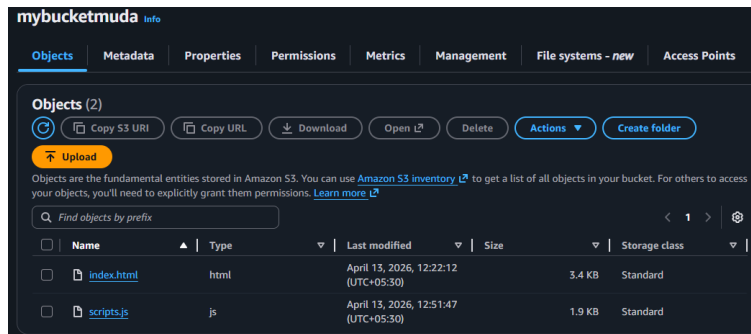


```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "PublicReadAccess",
6        "Effect": "Allow",
7        "Principal": "*",
8        "Action": "s3:GetObject",
9        "Resource": "arn:aws:s3:::mybucketmuda/*"
10     }
11   ]
12 }
```

Note:- Initially, public read access was enabled on the S3 bucket to allow users to access the static website.

Later, this public access was removed and replaced with CloudFront-based secure access.



Step 5: Two Lambda functions were created.

Lambda Function 1: getdata

Runtime

- Python

Region

- ap-south-1

Purpose

To retrieve all student records from DynamoDB.

Configuration

- Connected to DynamoDB table studentData
- Used scan() operation
- Returned data with CORS headers

Code

```
lambda_function.py X
lambda_function.py
1 import json
2 import boto3
3
4 def lambda_handler(event, context):
5     dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
6     table = dynamodb.Table('studentData')
7
8     response = table.scan()
9     data = response['Items']
10
11     while 'LastEvaluatedKey' in response:
12         response = table.scan(ExclusiveStartKey=response['LastEvaluatedKey'])
13         data.extend(response['Items'])
14
15     return {
16         'statusCode': 200,
17         'headers': {
18             'Access-Control-Allow-Origin': '*',
19             'Access-Control-Allow-Headers': 'Content-Type',
20             'Access-Control-Allow-Methods': 'GET,POST,OPTIONS'
21         },
22         'body': json.dumps(data)
23     }
```

Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)

Deploy and run test event.

Lambda Function 2: insertdata

Runtime

- Python

Region

- ap-south-1

Purpose

To insert student records into DynamoDB.

Configuration

- Connected to DynamoDB table studentData
- Accepted request body from API Gateway
- Used put_item() operation

Code

```
lambda_function.py x
lambda_function.py
1  import json
2  import boto3
3
4  dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
5  table = dynamodb.Table('studentData')
6
7  def lambda_handler(event, context):
8      body = json.loads(event['body'])
9
10     table.put_item(
11         Item={
12             'studentid': body['studentid'],
13             'name': body['name'],
14             'class': body['class'],
15             'age': body['age']
16         }
17     )
18
19     return {
20         'statusCode': 200,
21         'headers': {
22             'Access-Control-Allow-Origin': '*'
23         },
24         'body': json.dumps('Student data saved successfully!')
25     }
```

Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)

Lambda Test Event Configuration

To test the POST Lambda function, a test event was created.

Test Event

```
1 {
2   "httpMethod": "POST",
3   "body": "{\"studentid\":\"1\",\"name\":\"Mudasir\",\"class\":\"10\",\"age\":\"20\"}"
4 }
```

Purpose:

This test event matched the API Gateway proxy request format and allowed the POST Lambda to be tested correctly.

6. API Gateway Configuration

Amazon API Gateway was used to expose the backend as a REST API.

API Type

- REST API
- Name: student-api
- API end point(Regional)

Deploy API

- New stage
- Name (prod)
- GET
- POST
- OPTIONS

Enabled CORS for:

- GET
- POST
- OPTIONS

API Invoke URL

<https://x6g7ydaswa.execute-api.ap-south-1.amazonaws.com/prod>

```
scripts.js > onclick
1 // Add your API endpoint here
2 var API_ENDPOINT = "https://x6g7ydaswa.execute-api.ap-south-1.amazonaws.com/prod";
3
4 // AJAX POST request to save student data
```

This URL was copied and updated inside scripts.js.

Lambda Integration

- GET method connected to getdata Lambda
- POST method connected to insertdata Lambda

Proxy Integration:

Click on POST method, go to integration request, click on edit , enable the proxy integratio.

- The application was tested using the S3 static website URL before integrating CloudFront.

Save and View Student Data

Student ID:

Name:

Class:

Age:

[Save Student Data](#)

Student Data Saved!

[View all Students](#)

Student ID	Name	Class	Age
22	udta punjab	11	44

7. CloudFront Configuration:

Amazon CloudFront was configured to securely deliver the frontend over HTTPS.

Origin Type

- Amazon S3

Origin Domain

- S3 bucket endpoint selected as origin

Origin Path

- Left empty because index.html and scripts.js were uploaded in the bucket root

Origin settings:

- (use recommended default setting)

Customize cache setting:

Viewer Protocol Policy

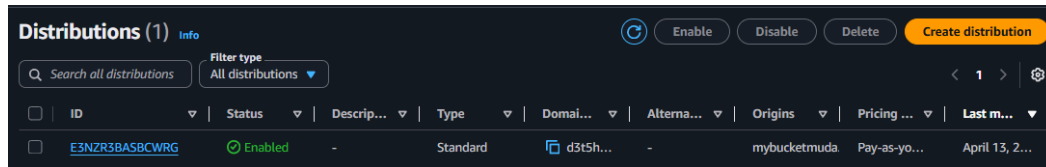
- Redirect HTTP to HTTPS

Allowed HTTP Methods

- GET

- HEAD
- OPTIONS
- PUT
- POST
- PATCH
- DELETE

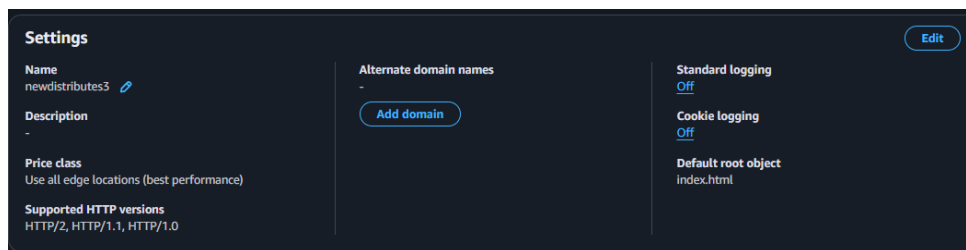
This was required because the frontend sends GET and POST requests.



Default Root Object: Go to general settings edit and give -- index.html

This was an important configuration because without the default root object, CloudFront returned

AccessDenied.



Note:- once you will create cloudfront distributions policy will be generated copy that and update S3 policy, or we can use policy generator.

Final CloudFront Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipalReadOnly",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3::mybucketmuda/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::356302822046:distribution/E3NZR3BASBCWRG"
        }
      }
    }
  ]
}
```

- Copy Distribution Domain name: d3t5hi6qkwddk8.cloudfront.net run on browser.

Save and View Student Data

Student ID:

Name:

Class:

Age:

[Save Student Data](#)

Student Data Saved!

[View all Students](#)

Student ID	Name	Class	Age
22	udta punjab	11	44
000010000000000000000000	bhat	MCA	99
kh	jl	l;kl;	;k890
1	Mudasir	10	20

Table: studentData - Items returned (5) [Actions](#) [Create Item](#)

Scan started on April 13, 2026, 22:29:51

☐	studentid (String)	age	class	name
☐	000010000000000000...	99	MCA	bhat
☐	kh	;k890	l;kl;	jl
☐	1	20	10	Mudasir
☐	00001	99	MCA	bhat
☐	student_id	age	student_class	name

Troubleshooting:

Issue 1: CORS Error

Solution: Enabled CORS in API Gateway and added headers in Lambda

Issue 2: POST Not Working

Solution: Enabled Lambda proxy integration

Issue 3: Data Not Showing

Solution: Parsed response.body in frontend

Issue 4: CloudFront AccessDenied

Solution: Fixed bucket policy and default root object

Issue 5: API Stage Mismatch

Solution: Updated correct stage in frontend

Conclusion:

This project successfully demonstrates a serverless web application using AWS services. The architecture eliminates the need for server management and provides scalability, security, and high availability. The integration of S3, CloudFront, API Gateway, Lambda, and DynamoDB represents a modern cloud-native application design.

